# Assignment 2: Jacobians

16-384: Robot Kinematics and Dynamics

Prof. Jeff Ichnowski
Albert Xu
Henry Liao
Karthik Srinivasan

# Contents

1 Theory Questions 3

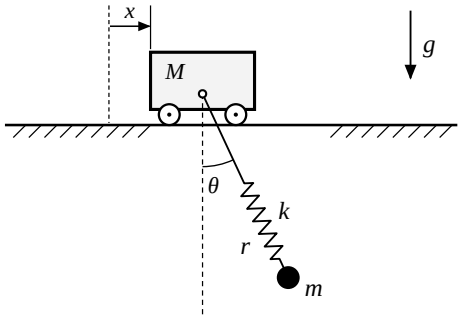
2 Code Questions 9

3 Submission Checklist 11

# 1 Theory Questions

1) Spring, cart, pendulum *30 points*

The following spring-cart-pendulum system shown below is a modification of the pendulum-cart discussed in class. The cart has mass $M$, the pendulum has mass $m$ and its spring has rest length $r_0$ and spring constant $k$.




a. [10 points] Write the potential and kinetic energy of the system in terms of the generalized coordinates $q = [x, r, \theta]^\top$ and their time derivatives $\dot{q} = [\dot{x}, \dot{r}, \dot{\theta}]^\top$. Then write the Lagrangian of the system.

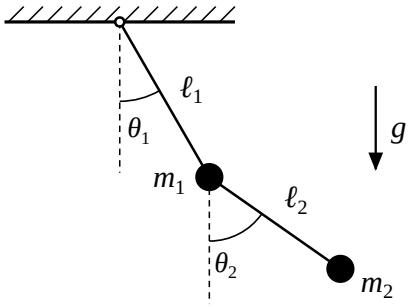
b. [10 points] Write the derivatives of $\mathcal{L}$ with respect to $q$ and $\dot{q}$ (6 in total).

c. [5 points] Write the equations of motion of the system using the Euler-Lagrange equations.

d. [5 points] Suppose you can control the cart with an external force $f_x$. Write the equations of motion with the control input $f_x$.

2) Double Pendulum *30 points*

The double pendulum is a classic example of a chaotic system, yet its equations of motion are relatively simple to derive. You will be using your derived equations of motion to simulate the double pendulum in the coding section of this assignment.



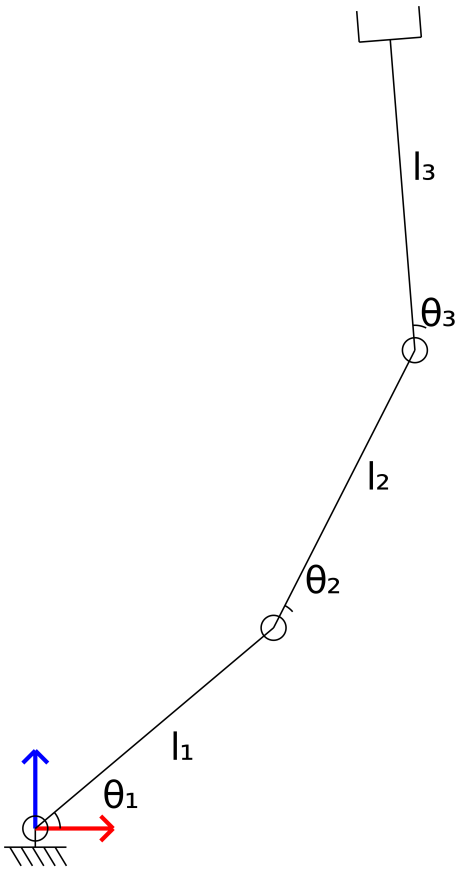

1. [10 points] Write the potential and kinetic energy of the system in terms of the generalized coordinates $q = [\theta_1, \theta_2]^\top$ and their time derivatives $\dot{q} = [\dot{\theta}_1, \dot{\theta}_2]^\top$.

2. [10 points] Write the derivatives of $\mathcal{L}$ with respect to $q$ and $\dot{q}$ (4 in total).

3. [10 points] Write the equations of motion of the system using the Euler-Lagrange equations.

3) RRR Arm Dynamics *30 points*

Consider the following robot. Assume no joint limits, and that the links lengths are $l_1 = l_2 = l_3 = 1$, and the masses are $m_1 = m_2 = m_3 = 1$. The joint angles are measured as a relative angle from the previous link / horizontal, so $\theta_1 = \theta_2 = \theta_3 = 0$ means the arm is fully extended to the right.




1. [10 points] Write the potential and kinetic energy of the system in terms of the generalized coordinates $q = [\theta_1, \theta_2, \theta_3]^\top$ and their time derivatives $\dot{q} = [\dot{\theta}_1, \dot{\theta}_2, \dot{\theta}_3]^\top$.

2. [10 points] Write the derivatives of $\mathcal{L}$ with respect to $q$ and $\dot{q}$ (6 in total).

3. [10 points] Write the equations of motion of the system using the Euler-Lagrange equations. Include external torques $\tau_1, \tau_2, \tau_3$ applied at each joint.

4. [5 points] Find the standard form matrices $M(q)$, $C(q, \dot{q})$, and $G(q)$ such that the equations of motion can be written as $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$.

5. [5 points] What are the joint torques $\tau_1, \tau_2, \tau_3$ required to hold the arm in the configuration $\theta_1 = \theta_2 = \theta_3 = 0$ (fully extended to the right) with no motion? How does this compare to the Jacobian-based gravity compensation method discussed previously?

# 2 Code Questions

1) Double Pendulum Simulation *45 points*

In this question you will simulate the double pendulum from the written section using the equations of motion you derived. All the code is in the `code/` folder; `code/README.md` explains how to run each script. You edit and submit only `double_pendulum.py`.

Use the same conventions as the written section: $q = [\theta_1, \theta_2]^\top$ with both angles measured from the downward vertical (counterclockwise positive), and point masses $m_1, m_2$ on massless links $\ell_1, \ell_2$. There is no applied torque in the code ($\tau = 0$). In your code, use the constants `M1`, `M2`, `L1`, `L2` and `G` at the top of `double_pendulum.py` rather than writing the numbers in directly, because the autograder changes them.

a. [20 points] Implement `dynamics(q, qd)`, which returns the joint accelerations $\ddot{q}$ as an array of shape $(2,)$. Your Euler-Lagrange equations are linear in $\ddot{q}$, so write them as $M(q)\ddot{q} = b(q, \dot{q})$ and solve for $\ddot{q}$ with `np.linalg.solve`.

b. [15 points] Implement `simulate(q0, qd0, T, dt)`. We recommend using a library like `scipy.integrate.solve_ivp` to integrate the equations of motion with the supplied `RTOL` and `ATOL` tolerances, but also encourage you to try implementing your own integrator to learn a little about numerical integration. You'll see the difference in (c).

The output format should be a tuple `(t, q, qd)` of numpy arrays, sampled every `dt` seconds from $0$ to $T$ inclusive (use `np.arange(0, T + dt / 2, dt)`, so there are $N = T/\Delta t + 1$ samples):

- `t`: the sample times $0, \Delta t, 2\Delta t, \ldots, T$, shape $(N,)$.
- `q`: the joint angles $[\theta_1, \theta_2]$ at each sample time, shape $(N, 2)$.
- `qd`: the joint velocities $[\dot{\theta}_1, \dot{\theta}_2]$ at each sample time, shape $(N, 2)$.

The first row of `q` and `qd` is the initial state `q0`, `qd0`.

Run `python autograder.py` to check parts (a) and (b), then watch your pendulum with `python visualize.py`.

c. [5 points] With no friction and no applied torque, the total energy $E = T + V$ of the double pendulum stays constant. That gives you a way to check a simulation without knowing the right answer: if $E$ changes over time, the simulation is not exactly following the physics.

Implement `total_energy(q, qd)` in `double_pendulum.py` using your $T$ and $V$ from the written section (the autograder checks it too). Then run `python energy_check.py`, which simulates the pendulum for 20 seconds and plots its total energy over time.

Does your simulation conserve energy? If the energy changes noticeably, what could cause it?

d. [5 points] Run `python visualize.py --chaos 5 --plot`, which simulates five pendulums whose starting $\theta_2$ values differ by only $0.001$ rad. About how long do the pendulums stay together before they visibly separate? Now try a small initial angle, `python visualize.py --chaos 5 --q0 10 5`. Is the motion still chaotic? Explain the difference.

# 3 Submission Checklist

- ☐ Create a PDF of your answers to the written questions with the name `<andrew_id>.pdf` and submit it to Gradescope.
- ☐ Run `autograder.py` and confirm it reports no failures.
- ☐ Run `create_submission.py` in the python terminal.
- ☐ Rename `submission.zip` and upload `<andrew_id>.zip` to Gradescope.
- ☐ Coding questions are autograded, with no limit on number of submissions within the mandated deadline.